

Lab 2 Extra

准备工作：创建并切换到 lab2-extra 分支

请在**自动初始化分支**后，在开发机依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab2-extra
```

初始化的 lab2-extra 分支基于课下完成的 lab2 分支，并且在 tests 目录下添加了 lab2_selfmap 样例测试目录。

题目背景

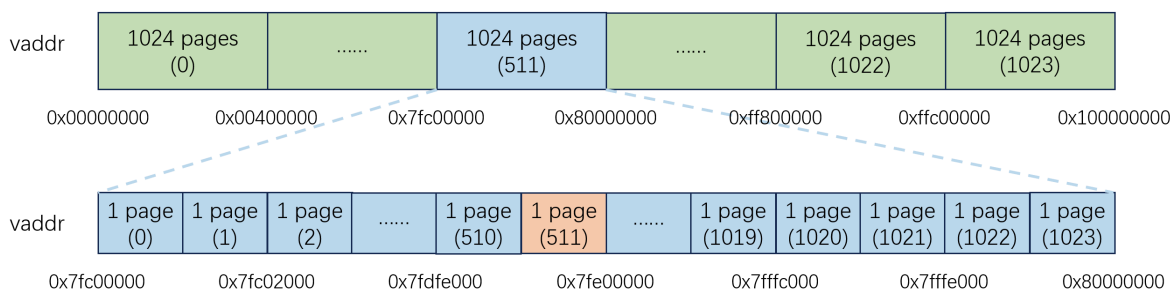
自映射是我们 MOS 中的一大特色，也是理论考试中经常会出现的重难点。在本次实验中，我们将会通过一段简单的讲解，以及一道有趣的题目，带大家深入理解自映射的作用、原理和实现方法。

如果你确信自己已经充分理解了自映射，那么可以跳过本章的内容，直接阅读**题目描述**一章。

我们知道，在 MOS 的两级页表设计中，每个进程将 1 个 4 KB 的页面作为页目录，页目录中有 1024 个 4 Bytes 的页目录项，分别指向 1024 张页表。每张页表依然占据 1 个 4 KB 的页面，每张页表中有 1024 个 4 Bytes 的页表项，分别指向 1024 个物理页面。这样，我们就能够通过页表查询到 $1024 * 1024 * 4KB = 4GB$ ，即 32 位的虚拟地址空间对应的物理页面。

在目前的 MOS 中，我们只能通过访问 kseg0 段的虚拟地址来访问页表，这就限制了我们只能在内核态访问它们。在之后的实验中，我们希望能够为用户态以只读的模式访问到页表，这样就可以将许多本来需要在内核态才能完成的操作移至用户态，体现微内核的思想。同时，不允许在用户态修改页表的内容，也保证了操作系统内核的安全。

那么，如果我们想要在用户态读取到页表中的内容，就需要使用 kuseg 段的虚拟地址与页表的物理页面之间建立映射。于是我们考虑划出一个 $1024 * 4KB = 4MB$ 大小的虚拟地址空间，将其依次与 1024 张页表建立映射。我们假设这段虚拟地址是 `[0x7fc00000, 0x80000000)`，于是它们的映射关系如下图所示：

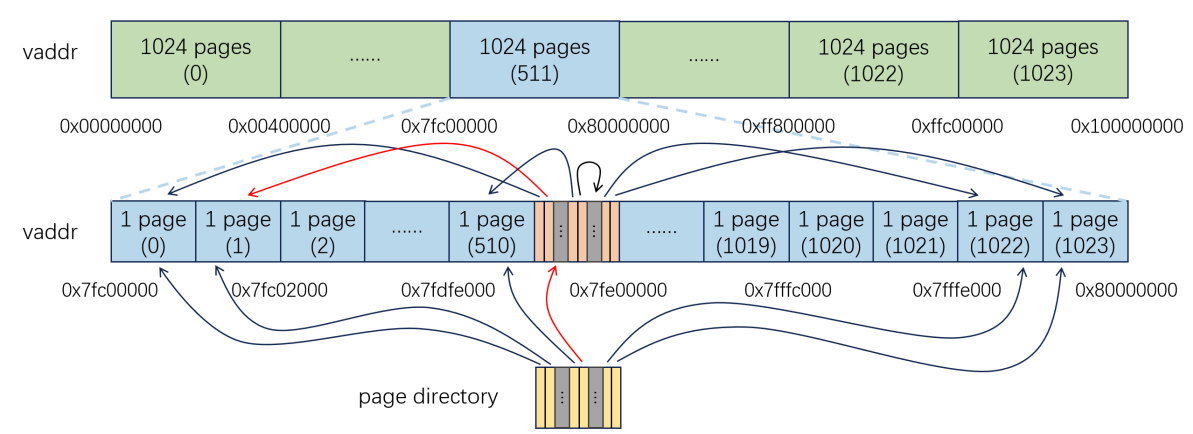


(如果你无法准确分辨图片中的颜色，请及时联系助教，助教可以帮你指出图中的颜色)

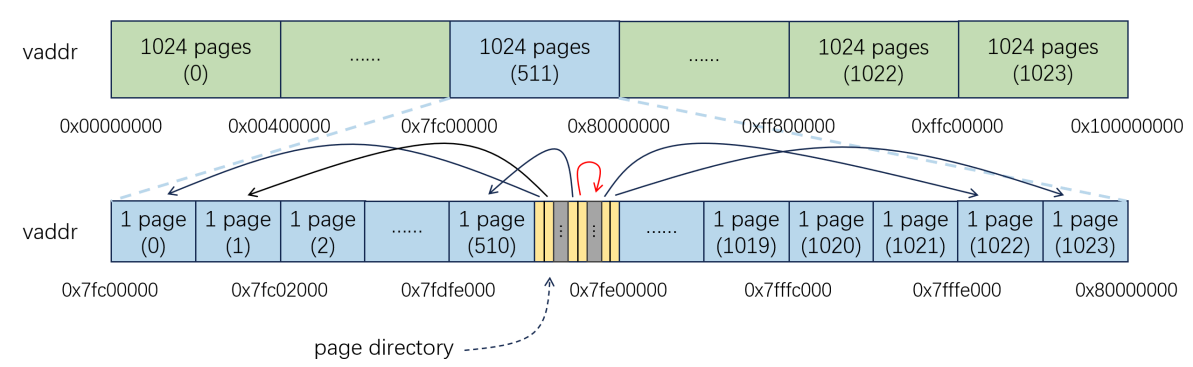
图中上方的条形图为整个 4GB 的虚拟地址空间，其中蓝色的方块代表着 1024 张页表对应的 4MB 虚拟地址空间。我们将这部分虚拟空间放大来看，就得到了下方的条形图，其中每个方块代表着 1 张页表对应的 4KB 虚拟地址空间。

我们来分析一下此时该如何通过 kuseg 段虚拟地址访问到任意页表。我们首先给出要访问的页表的虚拟地址，显然这个虚拟地址一定位于 `[0x7fc00000, 0x80000000)` 范围内，即 4GB 虚拟地址空间中的第 511 个 4MB 虚拟地址空间（从 0 计数，下同），于是我们找到页目录（图中金色的部分）中的第 511 条页目录项，并据此找到第 511 张页表（图中橙色的部分）。

接下来根据要访问的页表在 kuseg 段的虚拟地址，我们在橙色页表中寻找对应的页表项，并据此找到对应的页表。假设我们要访问的是第 1 张页表，那么上述过程就如下图红色箭头所示：



我们知道，操作系统在启动的时候不会一次性创建全部的 1024 张页表，而是在程序运行的过程中动态创建。从上述过程中我们能够看出，如果需要创建一个新页表，我们原本只需要维护页目录中指向新页表的页目录项，现在则还需要再维护橙色页表中指向新页表的页表项，这样才能够保证我们可以使用 kuseg 段虚拟地址访问到页表。然而从上图中我们可以看出，页目录和橙色页表的所有表项实际上都是完全相同的，于是我们就可以直接用页目录替换掉橙色页表，这就是所谓的自映射：



在之前的设计中，页目录的第 511 条页目录项指向橙色页面。经过替换之后，页目录的第 511 条页目录项就指向了自己。也就是说，这个页面既是页目录，又是 1024 张页表中的第 511 张页表，真的是太美妙了！

到这里，相信你已经充分理解了自映射的含义，让我们开始接下来的挑战吧！

提示：在使用页目录替换掉橙色页表之后，我们就不再需要额外维护原本橙色页表中的页表项了。不过，在建立自映射的时候，你还需要额外维护 1 个页目录项，以确保页目录的某条表项指向自己。

题目描述

在本题中，你需要在 MOS 中模拟自映射的建立、查询与解除。具体来说，你需要实现 `create_self_map()` `pte_va()` `pde_va()` `remove_all_self_map()` 4 个函数。

自映射的建立

`create_self_map()` 函数的功能为：给定页目录首地址 `pgdir` 和 `asid`，将所有页表自映射至 `[base, base + 1024 * PAGE_SIZE)` 的连续 4MB 虚拟地址空间。你可以参考如下实现流程：

1. 遍历 `[base, base + 1024 * PAGE_SIZE)` 的虚拟地址，检查其是否已经与物理页面建立了映射。如果已经建立了映射，则使用 `page_remove()` 函数解除映射，并使用计数变量 `count` 记录已经解除的映射数量；
2. 将所有页表自映射至 `[base, base + 1024 * PAGE_SIZE)` 的地址空间。这里要注意的是，你需要手动为页目录项加上合适的权限位，保证其有效且不可写，如果你在实现这一步时遇到了困难，请仔细阅读**题目背景**一章的内容；
3. 返回计数变量 `count` 的值。

自映射的查询

`pte_va()` 函数的功能为：给定自映射虚拟地址空间的首地址 `base`，计算使用自映射访问第 `pdeno` 张页表的第 `pteno` 个页表项时，**需要使用的虚拟地址**。

`pde_va()` 函数的功能为：给定自映射虚拟地址空间的首地址 `base`，计算使用自映射访问页目录中的第 `pdeno` 个页目录项时，**需要使用的虚拟地址**。

这两个函数请你根据对自映射的理解自行实现，在这里不给出过多的提示。使用合适的位运算，你可以分别使用一行代码完成这两个函数。

自映射的解除

`remove_all_self_map()` 函数的功能为：给定页目录首地址 `pgdir` 和 `asid`，解除当前已经建立的**所有**自映射。你可以参考如下实现流程：

1. 遍历所有页目录项，找到其中用于实现自映射的表项，将表项清空以解除自映射，并使用计数变量 `count` 记录已经解除的自映射数量，如果你在实现这一步时遇到了困难，请仔细阅读**题目背景**一章的内容；
2. 通过上述表项的地址相对于页目录首地址的偏移，计算自映射虚拟地址空间的首地址 `base`；
3. 对于每一个解除的自映射，遍历 `[base, base + 1024 * PAGE_SIZE)` 的虚拟地址，清空其在 TLB 中的缓存；
4. 返回计数变量 `count` 的值。

提示 & 注意事项

1. 在整道题目中，我们均**不考虑**页控制块（即 `Page` 结构体）中 `pp_ref` 字段的变化。
2. 由于我们尚未实现进程管理与用户态，在本题中，我们认为整个操作系统中只存在 1 张页目录，其基地址即为函数中的参数 `Pde *pgdir`。另外注意，请不要使用 `cur_pgdir` 这个全局变量。
3. 在测试数据中，我们保证不会出现两次自映射虚拟地址空间相同的情况。不过在自映射解除之后，其虚拟地址依然有可能与其它物理页面建立映射，你需要维护好 TLB 表项。
4. 在测试数据中，我们保证 `pgdir` 为正确的页目录首地址，保证 `asid` 为 0 到 255 之间的整数，保证 `base` 在 `kuseg` 段范围内且为 `1024 * PAGE_SIZE` 的倍数，保证 `pdeno` 和 `pteno` 为 0 到 1023 之间的整数。

实验提供代码

请将本部分提供代码附加在你的 `include/pmap.h` 中：

```
int create_self_map(Pde *pgdir, u_int asid, u_long base);
u_long pte_va(u_long base, u_int pdeno, u_int pteno);
u_long pde_va(u_long base, u_int pdeno);
int remove_all_self_map(Pde *pgdir, u_int asid);
```

请将本部分提供代码附加在你的 `kern/pmap.c` 的尾部，然后开始完成题目。

```
int create_self_map(Pde *pgdir, u_int asid, u_long base) {
    int count = 0;
    /* Your Code Here (1/4) */
    return count;
}

u_long pte_va(u_long base, u_int pdeno, u_int pteno) {
    /* Your Code Here (2/4) */
}

u_long pde_va(u_long base, u_int pdeno) {
    /* Your Code Here (3/4) */
}

int remove_all_self_map(Pde *pgdir, u_int asid) {
    int count = 0;
    /* Your Code Here (4/4) */
    return count;
}
```

本地测试说明

你可以使用：

- `make test lab=2_selfmap && make run` 在本地测试上述样例（调试模式）
- `MOS_PROFILE=release make test lab=2_selfmap && make run` 在本地测试上述样例（开启优化）

或者在 `init/init.c` 的 `mips_init` 函数中自行编写测试代码并使用 `make && make run` 测试。

如果样例测试中输出了如下结果，说明你通过了本地测试。

```
Congratulations! You passed the local test!
```

提示：本地测试为综合测试，即使无法通过本地测试，只要有部分功能是正确的，也有可能通过线上评测的部分测试点，请积极尝试提交，尽量提高自己的分数！

提交评测

请在开发机中执行下列命令后，在课程网站上提交评测。

```
$ cd ~/学号/
$ git add -A
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push
```

评测说明

评测时使用的 `mips_init()` 函数示意如下：

```
void mips_init() {
    mips_detect_memory();
    mips_vm_init();
    page_init();

    test_selfmap();

    halt();
}
```

`test_selfmap()` 函数会被测试文件中的函数替代，请不要修改其中的内容。

具体要求和分数分布如下：

测试点序号	评测说明	分值
1	检查 <code>pte_va()</code> 函数和 <code>pde_va()</code> 函数的计算结果	30
2	检查 <code>create_self_map()</code> 函数中原有映射的解除与自映射的建立	20
3	检查 <code>remove_all_self_map()</code> 函数中自映射的解除与 TLB 的维护	20
4	检查 <code>create_self_map()</code> 函数和 <code>remove_all_self_map()</code> 函数的返回值	10
5	综合测试	20

测试点依赖关系如下（箭头方向表示依赖方向）：

